

Race Condition pada Penjualan Stok – Dari Bug Sepele ke Rusaknya Invariant

Subtitle: Analisis kasus bengkel dengan stok 1 unit, dua kasir, dan bagaimana pola READ → CHECK → WRITE yang tidak atomic menghancurkan invariant bisnis.

Permasalahan

Toko bengkel punya 1 unit Oli Mesin. Dua kasir menekan "Bayar" hampir bersamaan. Hasil: 2 penjualan berhasil, stok akhir 0. Invariant `initial == success + final` menjadi `1 != 2 + 0`.

Ini bukan teori – Lab 05 mereproduksinya secara deterministik dengan channel barrier dan 500 goroutine.

Pola READ → CHECK → WRITE

Setiap logika bisnis yang membaca, memeriksa, lalu menulis state yang dibagi mengikuti pola ini:

```
...  
READ stock  
CHECK stock > 0?  
WRITE new stock  
...
```

Jeda antara CHECK dan WRITE adalah jendela race. Pada sistem konkuren, request lain bisa mengubah state di antara langkah-langkah tersebut. Yang menarik: `go test -race` tidak mendeteksi business race condition ini karena variabel di memory sudah thread-safe.

Bukti Deterministik: TestLostUpdate_Deterministic

Test di `lost\_update\_test.go` memaksa urutan eksekusi: A READ → B READ → A WRITE → B WRITE. Test PASS berarti invariant hancur – memang desain untuk menguji implementasi unsafe.

```
...go  
// UnsafeInventory: READ → CHECK → WRITE tidak atomic  
func (r *UnsafeInventory) TrySell(ctx context.Context, productID string) error {  
    stock, _ := r.GetStock(ctx, productID)  
    if stock <= 0 {  
        return ErrOutOfStock  
    }  
    return r.SetStock(ctx, productID, stock-1)  
}
```

Solusi 1: Atomic Conditional Update

Satu statement SQL: `UPDATE inventory\_products SET stock = stock - 1 WHERE id = \$1 AND stock > 0 RETURNING stock`. 1 row affected = sukses, 0 rows = out of stock. Tanpa SELECT sebelum UPDATE, tanpa stale read window.

Bukti: `TestAtomicUpdate\_HighContention` – 500 goroutine, stok awal 100, success=100, rejected=400, final=0. Invariant holds.

Solusi 2: Row Lock (Pessimistic)

`SELECT ... FOR UPDATE` mengambil row-level lock. Transaction B diblokir sampai A commit. `TestPostgresRowLock\_ConcurrentStock` membuktikan blocking via `pg\_stat\_activity` – B tercatat dalam keadaan `wait\_event\_type = 'Lock'` sebelum A commit.

Solusi 3: UNIQUE Constraint

Untuk booking slot, `UNIQUE(branch\_id, service\_date, slot\_time)` adalah safety net terakhir. `TestPostgres\_ConcurrentBooking`: 500 goroutine, created=1, conflict (SQLSTATE 23505)=499. Database engine yang menolak duplikat – bahkan jika ada bug di aplikasi.

Transaction Saja Tidak Cukup

Pada `READ COMMITTED`, dua transaksi bisa baca stock = 1 bersamaan, keduanya update ke 0. Correctness bergantung pada pola query, lock, dan isolation semantics – bukan hanya BEGIN/COMMIT.

Kesalahan Umum

- Validasi hanya di frontend (bypass via API).
- Tidak pakai UNIQUE constraint.
- Menganggap BEGIN/COMMIT cukup.
- Mutex untuk microservices (hanya mengunci satu instance).

Referensi Implementasi

Source code lengkap ada di repository Software Engineering Lab.

<https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/05-race-condition>